INTRODUCTION TO EMBEDDED SYSTEMS & IoT DEVICE DESIGN

EMBEDDED SYSTEMS TASK – 1

Submitted by : RANDRIAMIHAJA TIANA ELI THONNY
Department : ELECTRICAL AND ELECTRONIS ENGINEERING
College: ANDHRA UNIVERSITY
Date: 16 October 2026

Selected mini project: Motion Detection Alarm using Arduino and a PIR motion sensor.

Project type: Simulation design. The report includes a wiring diagram, code, expected behaviour and testing checklist.

1. INTRODUCTION

1.1 What is an embedded system?

An embedded system is a computer-based system designed to perform a specific task as part of a larger device. It combines hardware and software. The hardware may include a microcontroller, memory, sensors and output devices, while software controls how the system responds to inputs. Embedded systems are commonly designed to be compact, reliable and energy-efficient.

1.2 REAL-WORLD APPLICATIONS

Smart home: Automatic lights, smart locks, temperature control and security alarms.

Automotive: Engine control, airbag systems, parking sensors and anti-lock braking systems.

Healthcare: Digital thermometers, patient monitors, infusion pumps and wearable health trackers.

Robotics: Motor control, obstacle detection, navigation and automated movement. **Industry:** Process monitoring, machine control and safety systems.

1.3

Feature	Microcontroller (MCU)	Microprocessor (MPU)
Integration	CPU, memory and peripherals on one chip	Mainly CPU; often uses external memory/peripherals
Typical use	Dedicated control tasks	General-purpose computing and complex processing
Power/cost	Often lower power and lower cost	Often higher power and system cost
Examples	ATmega328P (Arduino Uno), ESP32, RP2040	

MICROCONTROLLER VS. MICROPROCESSOR:

A microcontroller is suitable for small control projects because it integrates the main parts needed to read inputs and control outputs.

A microprocessor is often used where greater computing performance and a larger operating system are needed.

2. COMPONENT STUDY

2.1 MICROCONTROLLER PLATFORMS

Arduino Uno: A beginner-friendly development board used to learn electronics and control LEDs, sensors and motors. Its ATmega328P microcontroller reads input pins and controls output pins.

ESP32: A microcontroller family with Wi-Fi and Bluetooth on many variants, making it useful for connected IoT devices and remote monitoring.

Raspberry Pi Pico: A compact board based on the RP2040 microcontroller. It supports GPIO, PWM and common communication interfaces. The standard Pico does not have built-in Wi-Fi; Pico W variants provide wireless connectivity.

2.2 BASIC SENSORS

Temperature sensor: Measures temperature and provides an electrical signal or digital reading. Used in room monitoring, HVAC and equipment protection.

Motion sensor: A PIR (passive infrared) sensor detects changes in infrared energy, such as those caused by a person moving across its field of view. Used in alarms and automatic lighting.

Light sensor: An LDR changes resistance according to the amount of light falling on it. A circuit or microcontroller can use the reading to control lighting.

3. MINI PROJECT – MOTION DETECTION ALARM

3.1 AIM

To design a simple motion detection alarm in which a PIR sensor detects movement and activates an LED and buzzer. The project can be built using real hardware or tested in a simulator.

3.2 COMPONENTS REQUIRED

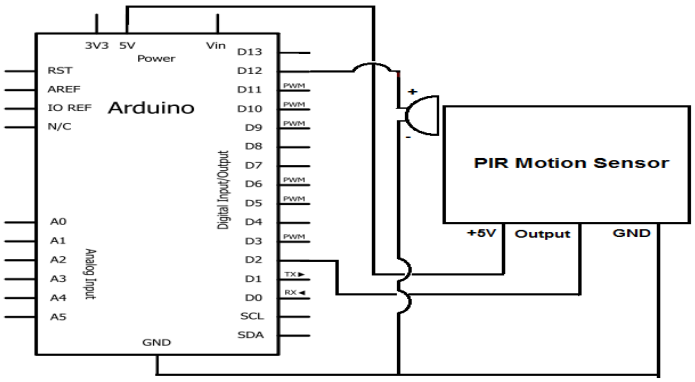
Arduino Uno; PIR motion sensor (such as HC-SR501); LED; 220 Ω resistor for the LED; active buzzer; breadboard; jumper wires; USB power supply; and a simulator if hardware is not available.

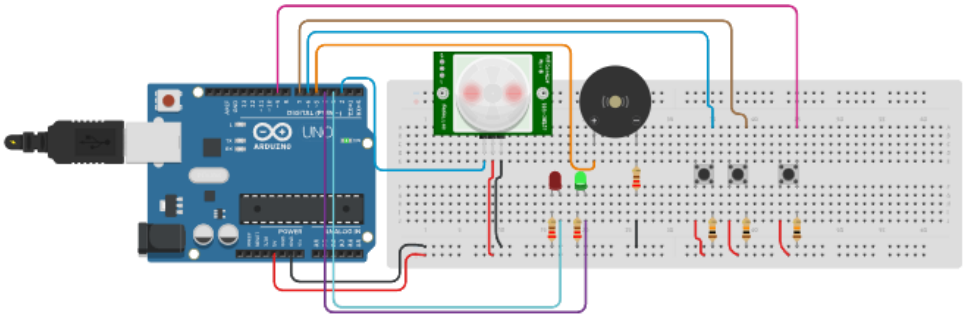
3.3 WORKING PRINCIPLE

The PIR sensor monitors changes in infrared radiation. When it detects motion, its OUT pin goes HIGH. The Arduino reads this digital input and turns on the LED and buzzer. When the sensor output returns LOW, the Arduino turns them off. The sensor may need a short warm-up period after power is applied.

3.4 CIRCUIT DIAGRAM AND CONNECTIONS

Illustrative wiring diagram





Component pin	Connection

PIR VCC	Arduino 5V (check sensor voltage rating)
PIR GND	Arduino GND
PIR OUT	Arduino digital pin 2
LED anode (+)	Arduino digital pin 8 through a 220 Ω resistor
LED cathode (−)	Arduino GND
Active buzzer (+)	Arduino digital pin 9; use a transistor driver if the buzzer current exceeds safe pin limits
Active buzzer (−)	Arduino GND

3.5 ARDUINO/C++ CODE

```
const int pirPin = 2;
const int ledPin = 8;
const int buzzerPin = 9;

void setup() {
pinMode(pirPin, INPUT);
pinMode(ledPin, OUTPUT);
pinMode(buzzerPin, OUTPUT);
Serial.begin(9600); }

void loop() {
  int motion = digitalRead(pirPin);

  if (motion == HIGH) {
digitalWrite(ledPin, HIGH);
digitalWrite(buzzerPin, HIGH);
Serial.println("Motion detected!");   }
else {
    digitalWrite(ledPin, LOW);
digitalWrite(buzzerPin, LOW);
Serial.println("No motion");   }

  delay(200);
}
```

3.6 CODE EXPLANATION

The pin variables identify the sensor, LED and buzzer connections. In setup(), the sensor pin is configured as an input and the LED and buzzer pins as outputs. In loop(), digitalRead() checks the PIR output. If it is HIGH, the program switches on the LED and buzzer; otherwise, it switches them off. Serial messages help the user observe the sensor state.

3.7 EXPECTED RESULT AND TEST CHECKLIST

Expected behaviour: when movement is detected in front of the PIR sensor, the LED lights and the buzzer sounds. When no movement is detected and the sensor output is LOW, both outputs remain off.

Test	Action	Expected observation
1	Power on; allow sensor to warm up	System becomes ready
2	Move in front of PIR sensor	LED turns on and buzzer sounds
Test	**Action**	**Expected observation**
3	Wait until sensor output goes LOW	LED and buzzer turn off

3.8 REAL-WORLD APPLICATIONS

Home and office security, automatic corridor lighting, motion-triggered warnings, restricted-area monitoring and energy-saving occupancy detection. Real security systems also need appropriate sensor placement, false-trigger handling, backup power and reliability measures.

4. CONCLUSION

This task introduces embedded systems, common microcontroller platforms and basic sensors.

The proposed motion detection alarm demonstrates how a sensor input can be processed by an Arduino and used to control output devices. The design is suitable for a beginner simulation and can be expanded with wireless alerts or data logging using an IoT-capable board such as an ESP32.